Silent Procedural Defects in Financial Tool Use: An Evaluation Framework, Offline Validation, and a Live Stress Test

Jasleen Sandhu

Abstract

A financial answer can be numerically correct and still break an evidence requirement. I call this a silent procedural defect and build a synthetic benchmark that scores answer correctness separately from retrieval and citation compliance. The questions ask for figures as originally reported, but the data tool returns restated figures by default. The primary measure is the silent-defect rate among outcome-correct runs, reported beside overall correctness. A deterministic checker is validated on planted defects and harmless reorderings, and three scripted policies show that the pipeline recovers behaviour I built in; they say nothing about language models. I then ran one model, `gpt-4o-mini`, on 100 cases under six conditions, in two repetitions of 1,200 runs. No silent defect occurred in any single-setback condition, and the combined condition produced one in each repetition, among 188 and 186 correct runs. Case-level 95% upper bounds were about 3% and 5%, but the cases draw on only 14 companies, and company-level bounds are about 31% and 49%. All eight defective runs fell back on the tool's restated default after failed calls. A second study lengthened the task to 2, 8 and 16 required retrievals (1,080 runs). Baseline stayed clean, but with failing tools 14 of 65 correct runs (21.5%, interval 12.0 to 31.9) were silent at 16 retrievals, and 5 of 51 (9.8%) with all setbacks combined, each citing a figure whose retrieval had failed. Outcome-only evaluation would have scored every one of these runs as a success. These results come from one small model on a synthetic task and do not show how often silent defects occur in financial agents generally.

1 Introduction

A financial question often asks for more than a calculation. It may also say which reporting basis to use, which fiscal period it concerns, and what evidence should back the answer. Take a question about operating margin as originally reported. If a tool returns restated figures by default, it can still give the right number when the records in question were never restated, and a check that looks only at the answer would accept the result even though the requested basis was never retrieved.

This gap is why I focus on *silent procedural defects*: breaches of an explicit process rule in runs whose final numerical answer is correct. By "silent" I mean undetected by outcome-only evaluation; the breach is still visible in a complete tool trace. I do not treat a process breach as a financial loss. The point is that the gap can be measured only when outcome correctness and process compliance are scored separately.

Earlier work has already shown that outcome-correct runs often fail process checks in customer-service and tool-use benchmarks, so I do not claim to have discovered the phenomenon. I ask two narrower questions about how to measure it. First, does a deterministic checker agree with independently written labels on planted defects, harmless reorderings and extra lookups? Offline, the pipeline has to recover behaviours I built myself, so this shows only that the checker works as

Preprint.

intended. Second, for one served model on a synthetic financial-statement task, what upper bound do the live data place on the run-level silent-defect rate $\sigma = \Pr(V = 1 \mid A = 1)$ under six operating conditions and at three task lengths, once uncertainty resamples the highest shared source group? I make no claim about how often live financial agents show the defect in general.

My contribution is an executable evaluation framework, a validation of its checker, and a live stress test of one model. A synthetic environment with a known answer oracle and a trace-based rubric separates cases where the choice of basis makes no numerical difference from cases where it changes the answer. The live study asks whether silent defects appear when tools fail, time is short, the request is buried, a plausible but unfiled data source is on offer, or the task is long. It does not test an intervention, several models, or the proposed accounts-receivable setting; those need further experiments (Section 6). Only the live evaluations give observations about a language model, and these hold only for that model, those synthetic worlds and the conditions studied.

In brief, the scripted controls show that the checker recovers constructed defects and that the mix of strata changes the pooled rate. For `gpt-4o-mini`, silent defects were absent at baseline and under each single setback, and rare (one in each repetition) when all four setbacks were combined, although the bounds are too wide to call the rate small. When the task was made longer, baseline stayed clean, but with failing tools about a fifth of correct runs at 16 retrievals were silent, because the model cited figures it had never retrieved. An outcome-only check would have scored every one of these runs as a success, which is the case for scoring correctness and compliance separately.

2 Related work

Agent benchmarks have examined task completion, adherence to domain rules and consistency over repeated trials. Yao et al. (2024) introduce τ-bench, where agents use simulated users and domain-specific tools under policy guidelines, and compare final database states with annotated goals. The closest work I found separates outcome from process in the same way. Yu et al. (2026) score 120 tasks in six regulated customer-service domains for seven models, and call a run that passes the outcome check but fails the process check a false pass; they report that false passes are 80.07% of outcome passes. Cao et al. (2026) argue that task-completion benchmarks hide "corrupt successes", and report that between 27% and 78% of benchmark-reported successes in τ-bench are corrupt for three models. I rely on the headline figures of these papers and do not compare their procedural categories with mine in detail. In the parts I read, neither resamples at the level of shared source records, and neither tests a deterministic checker against planted defects and harmless reorderings. My work differs in setting and scope: a synthetic financial retrieval task with exactly known evidence requirements, a conditional rate with an explicit denominator, and uncertainty from resampling cases or companies. My rubric is also narrower, so my rates should not be compared directly with theirs.

The contrast between outcomes and intermediate processes also appears in mathematical reasoning (Lightman et al., 2023; Uesato et al., 2022). Uesato et al. (2022) report that the best process-based results reduced reasoning error among solutions with correct final answers from 14.0% to 3.4%, a measurement conditional on a correct answer that is close in form to σ, though about reasoning steps and not observable tool calls. Finance-specific agent benchmarks report accuracy on realistic tasks; the Finance Agent Benchmark of Bigeard et al. (2025) poses 537 expert-written questions over SEC filings, and its best reported model reached 46.8% accuracy. My synthetic task is far easier, with baseline accuracy near 94%, so my rates describe a narrow setting. On uncertainty, Miller (2024) argue that standard errors for language-model evaluations should account for dependence among questions that share a source; my resampling of whole companies follows the same principle. ToolEmu and AgentDojo test agents against emulated tools and prompt injection (Ruan et al., 2023; Debenedetti et al., 2024), whereas the failure examined here needs no adversarial input: a documented tool default can disagree with the basis the question requests.

3 Methods

The study has three parts. A synthetic financial-retrieval benchmark supplies exact ground truth and a deterministic checker that scores answer correctness and procedural compliance separately. An offline validation checks the checker and the pipeline with scripted policies. A live evaluation then runs one served language model through the same tools, cases and rubric. Only the live evaluation

says anything about a language model. Its hypotheses, conditions and reporting rules were fixed in a written plan before the data were collected, whereas the company-level resampling and the trace checks were exploratory or post hoc, and I label them as such where they arise. Secondary analyses of repeat agreement, review cost and escalation are left out for space.

3.1 Environment, cases and rubric

The environment is synthetic, so records, required evidence and reference answers are known exactly. The agent has four tools: `get_company_profile`, `get_financials`, `get_restatement_notes` and `submit_answer`. The financial-data tool returns the restated basis by default, as its description states, while every question asks for figures as originally reported. An agent therefore only has to accept the default to fall into the trap, and nothing adversarial is needed to produce a silent defect.

Questions cover operating margin, net margin and revenue growth. Cases fall into two strata. In stratum A no year the question needs was restated, so a wrong-basis retrieval changes nothing numerically, and any resulting defect is silent by construction. Stratum B needs a restated year and a gap of at least 0.5 percentage points between the two bases, so a wrong basis gives a wrong answer. Strata are balanced by design, so the pooled rate depends on the chosen mix and not on how often restatements occur in practice. The offline validation uses 40 cases (20 per stratum) and the live evaluation 100 (50 per stratum), both generated with seed 0. The live cases draw on only 14 companies whose source records they share, which determines the unit of resampling below.

For reference answer y_i and submitted value $\widehat{y}_{ir}$ of run r on case i, correctness is $A_{ir} = \mathbb{1}\{|\widehat{y}_{ir} - y_i| \le 0.1\}$, with the tolerance in percentage points. This predicate contains no process control. Let R_i be the required evidence set, G_{ir} the items successfully retrieved and E_{ir} the items cited. The checker applies three rules: $M_{ir} = \mathbb{1}\{R_i \setminus G_{ir} \neq \varnothing\}$ (a required item was never retrieved), $W_{ir} = \mathbb{1}\{E_{ir} \setminus R_i \neq \varnothing\}$ (a cited item is not required) and $U_{ir} = \mathbb{1}\{E_{ir} \setminus G_{ir} \neq \varnothing\}$ (a cited item was never retrieved). The defect indicator is $V_{ir} = \mathbb{1}\{M_{ir} \vee W_{ir} \vee U_{ir}\}$ and the silent-defect indicator is $S_{ir} = A_{ir}V_{ir}$. A defect is a violated process rule and not demonstrated harm; in stratum A the basis error changes no number. The rubric is deliberately narrow. It covers retrieval and citation of evidence because those can be checked deterministically, and it does not cover authorisation, escalation or whether the agent reasoned from the wrong evidence.

3.2 Estimands and uncertainty

For a fixed agent configuration and case distribution, the primary estimand is the silent-defect rate conditional on outcome correctness,

$$\sigma = \Pr(V = 1 \mid A = 1) = \frac{\mathbb{E}[AV]}{\mathbb{E}[A]}, \qquad \Pr(A = 1) > 0, \tag{1}$$

estimated by the pooled ratio $\widehat{\sigma} = \sum_{i,r} A_{ir}V_{ir} \big/ \sum_{i,r} A_{ir}$ over complete runs. I always report its numerator and denominator beside the correctness rate and the unconditional silent-defect frequency. A run is complete if it has a final submission and no execution error, and incomplete runs are excluded from rates and counted. The conditional rate is undefined when a configuration has no correct run.

Cases, not runs, are the independent sampling units, because repeated runs of one case share its records. Intervals come from a percentile cluster bootstrap that resamples whole units (10,000 replicates, seed 12345). The offline controls resample cases. The live written plan also specified cases, but because the 100 cases share 14 companies I resampled companies instead, in line with clustering at the highest shared source group. This change was made after the data were seen. With 14 units the bootstrap is coarse and degenerate when a rate has zero or one events, so I also report two exact upper bounds. With C cases containing a correct run, e of them containing a silent run, K runs per case, N_c correct runs and p_{up} the exact one-sided 95% binomial upper bound, the case-level bound is $\sigma \le K\, p_{\mathrm{up}}\, C/N_c$. It treats cases as independent. The company-level bound is $\sigma \le g_{\mathrm{up}}\, G\, m/N_c$, where G is the number of companies with a correct run, g_{up} the exact bound for the companies containing a silent run, and m the most correct runs in any one company. It lets every correct run of an affected company be silent, so it is a conservative envelope and not an estimate.

3.3 Checker validation and scripted policies

The checker was validated independently of the agent. The repository holds 29 hand-built traces whose expected labels were written separately from the checker, covering planted defects, harmless reorderings, extra lookups and tolerance boundaries; the checker agrees with all 29. They were written in the same session as the checker, so they test the implementation against the stated rubric and not the rubric itself. After the live data were seen I also made eight edits to the real live traces, four planting a defect (for example removing a retrieval or citing the restated basis) and four harmless (for example reversing call order or adding a failed call), none changing the submitted value. An independent re-scoring from the raw logs found that four runs whose error was exactly 0.1 points had been scored incorrect by a floating-point comparison. I corrected this and report corrected counts throughout; no defect count changed.

Three scripted policies exercise the pipeline on the 40 offline cases, five runs each, giving 600 runs. The careful policy always requests the as-reported basis. The default-trap policy omits the basis argument and cites what it receives. The mixed-error policy requests the correct basis with probability 0.6 per retrieval, perturbs the fiscal year with probability 0.04 and attaches the as-reported label with probability 0.1 regardless of the basis returned. These probabilities describe the scripts and are not properties of any model.

3.4 Live evaluation

I ran one served model, `gpt-4o-mini` (`gpt-4o-mini-2024-07-18`), at temperature 1.0, with at most twelve tool-calling turns and 600 completion tokens per call. If the model replied without calling `submit_answer`, the harness sent up to two neutral reminders to submit. Six conditions were applied to the same 100 cases, each changing only the environment or the prompt so that reference answers, required evidence and the rubric stay identical. In `flaky`, each data call fails with a retryable 503 error with probability 0.25. In `pressure`, the prompt asks for as few billed calls as possible. In `noisy`, the question is buried in a message with irrelevant business detail. In `distractor`, an extra tool returns analyst estimates that are not filed figures. The `combined` condition applies all four, and `baseline` applies none. Each case was run twice per condition, giving 1,200 runs per repetition, and the whole suite was run twice. The first repetition is primary, and the second is reported separately and not pooled because both use the same cases.

The confirmatory hypothesis was that σ in each of the five non-baseline conditions exceeds σ at baseline, tested with a paired company bootstrap, one-sided and Holm-adjusted at level 0.05. The written plan was not independently registered or time-stamped, and it was prepared in the same session as a 40-case pilot whose cases are the first 40 of the 100 live cases. The numbers of cases and repetitions came from a design-effect heuristic and not from simulation.

3.5 Task-length ladder

The live suite used tasks of about three tool calls, which leaves little room to break a procedure. To test longer tasks I ran a second study with the same model, rubric and tolerance, following a written plan dated 29 October 2026 that, like the first, is not independently registered. Three tiers were generated: `short`, a single-year question (2 required retrievals); `medium`, the average of four yearly operating margins (8 retrievals); and `long`, the average of eight (16 retrievals). The longer tiers add a `calculate` tool that does arithmetic only, so errors reflect retrieval and not mental arithmetic. The world was regenerated with 60 companies and a restatement probability of 0.2, so the ladder is not directly comparable with the first suite. Each tier used 60 cases (30 per stratum) and two runs per case under the baseline, `flaky` and `combined` conditions, giving 1,080 runs. Step and token limits were raised for the longer tiers so that they did not truncate runs. Each ladder case uses one company, so companies are the independent units. The single confirmatory hypothesis was that σ rises with task length, tested within each condition as the long-minus-short difference with a company bootstrap, one-sided and Holm-adjusted across three conditions. Cells with fewer than 20 correct runs are described as barely estimable.

Table 1: Live evaluation of `gpt-4o-mini`, primary repetition: 100 cases, two runs per case, the same cases in every condition. Accuracy and $\widehat{\sigma}$ are pooled run-level rates with 95% intervals that resample whole companies (14 companies). Bound_c and Bound_g are the case-level and company-level 95% upper bounds on σ (percent); Bound_g is a conservative envelope and not an estimate. The second repetition is in the text.

Condition	Accuracy (%)	Defective	Silent/correct	$\widehat{\sigma}$ (%)	Bound_c / Bound_g	Calls/run
`baseline`	94.5 (89.7–98.2)	0	0/189	0.0 (0.0–0.0)	3.1 / 31.4	3.03
`flaky`	94.5 (90.0–98.1)	0	0/189	0.0 (0.0–0.0)	3.1 / 31.4	4.41
`pressure`	95.0 (90.4–98.9)	0	0/190	0.0 (0.0–0.0)	3.1 / 31.2	3.00
`noisy`	93.5 (88.2–97.9)	0	0/187	0.0 (0.0–0.0)	3.2 / 31.7	3.00
`distractor`	95.0 (89.2–99.1)	0	0/190	0.0 (0.0–0.0)	3.1 / 31.2	3.01
`combined`	94.0 (90.2–97.4)	4	1/188	0.5 (0.0–1.6)	5.0 / 48.6	4.37

4 Results

4.1 Offline validation with scripted policies

All 600 scripted runs were complete, and 14 automated tests of the generator, checker and reporting passed. The results show that the pipeline recovers the behaviour built into each script; they say nothing about language models. In stratum A, the careful policy produced 100 correct runs and no silent defects. The default-trap policy produced 100 correct runs, all silently defective. The mixed-error policy produced 95 correct runs, 62 of them silently defective ($\widehat{\sigma} = 65.3\%$, 95% CI 55.9–74.2). In stratum B, the careful policy again had no defects, and the default-trap policy produced no correct run, so its conditional rate is undefined and not zero. The mixed-error policy produced 41 correct runs, six silently defective ($\widehat{\sigma} = 14.6\%$, 2.5–29.0). Reporting the strata separately matters, since the pooled conditional rate for the mixed-error policy (50.0%, 39.1–59.6) depends on the chosen mix. The checker agreed with all 29 hand-built traces. On the edited real live traces it produced the expected flags for every applicable edit, with no disagreement in either repetition (8,962 and 8,950 edited traces). The expectations were written by me, so this tests the implementation on real trace structure and is not a blind human audit.

4.2 Live evaluation of `gpt-4o-mini`

All 1,200 runs of the primary repetition were complete. In the second repetition 1,199 were, and one combined-condition run without a valid submission was excluded from rates. Table 1 gives the primary repetition. At baseline accuracy was 94.5% (89.7–98.2) and no run broke the rubric, so $\widehat{\sigma}$ was 0 of 189 correct runs in the primary repetition and 0 of 190 in the second. The 21 incorrect baseline runs, 11 in the primary repetition and 10 in the second, were all process-compliant revenue-growth questions: the model retrieved and cited the required as-reported items but submitted a different value, which is consistent with arithmetic error and not procedural failure.

No single-setback condition produced a silent defect in either repetition, among 945 and 944 correct runs. The case-level upper bound was about 3% in each, but it treats the 100 cases as independent although they draw on 14 companies, and the company-level bound of 31–32% means the data cannot exclude much larger rates tied to a company-level mechanism. Failed calls alone did not degrade outcomes: under `flaky`, 19–20% of calls failed and calls per run rose from about 3.0 to 4.4, yet accuracy stayed at 94.5% and 93.5%. Silent defects occurred only in the `combined` condition, once in each repetition ($\widehat{\sigma} = 0.5\%$; case-level bound 5.0%, company-level 48.6% and 49.1%). Eight runs were defective in total, two of them silent. Every one fetched the tool's restated default after failed calls, and seven followed five or more failed calls. The two silent defects were in stratum A and cited restated items for a question that asked for as-reported figures; the six loud ones were in stratum B and gave wrong answers. None of the five confirmatory contrasts was supported (Holm-adjusted $p = 1.0$ throughout).

Table 2: Task-length ladder for `gpt-4o-mini`: 60 cases per tier, two runs per case. $\widehat{\sigma}$ and its 95% interval resample whole companies; Bound_c and Bound_g are the case-level and company-level upper bounds on σ (percent). Accuracy falls under failures at the long tier, so σ there rests on 51 to 65 correct runs. This is a different world from Table 1.

Tier	Condition	Calls	Acc. (%)	Silent/correct	$\widehat{\sigma}$ (%)	95% CI	Bound_c	Bound_g
short	baseline	3.0	93.3	0/112	0.0	0.0–0.0	5.2	20.6
short	flaky	4.5	93.3	0/111	0.0	0.0–0.0	5.3	20.8
short	combined	4.5	89.2	0/107	0.0	0.0–0.0	5.5	21.5
medium	baseline	11.3	91.7	0/110	0.0	0.0–0.0	5.3	20.9
medium	flaky	18.1	75.0	0/90	0.0	0.0–0.0	6.5	19.2
medium	combined	16.7	79.2	0/95	0.0	0.0–0.0	6.1	21.2
long	baseline	18.6	94.2	0/113	0.0	0.0–0.0	5.2	30.0
long	flaky	30.1	54.2	14/65	21.5	12.0–31.9	54.9	100.0
long	combined	28.9	42.5	5/51	9.8	2.6–18.2	38.5	90.0

4.3 Task length and silent defects

Table 2 gives the ladder. At baseline there was no silent defect at any task length, including 0 of 113 correct runs at 16 retrievals (bounds 5.2% and 30.0%), and none appeared in any condition on short or medium tasks.

Silent defects appeared only on long tasks with failing tools. Under `flaky`, 14 of 65 correct runs were silent ($\widehat{\sigma} = 21.5\%$, 95% interval 12.0 to 31.9), and under `combined` 5 of 51 (9.8%, 2.6 to 18.2), while accuracy fell to 54.2% and 42.5%. The confirmatory contrasts supported the hypothesis for both: the long-minus-short difference in σ was $+21.5$ points (11.5 to 32.1, Holm-adjusted $p < 0.001$) for `flaky` and $+9.8$ (2.8 to 18.2, $p = 0.009$) for `combined`, and zero at baseline. The company-level bounds in these cells were 100% and 90%, so the data fix the direction of the effect more firmly than its size. The unconditional defect rate rises the same way, from 1 to 7 to 56 defective runs of about 120 under `flaky` as tasks lengthen, so the rise is not an artefact of conditioning on correctness.

In all 19 silent long-tier runs the model cited at least one required figure it never retrieved, every attempt to fetch it having returned an error; 18 were missing one figure and one was missing two. None fetched the figure on the restated basis instead. In a post hoc tabulation, runs where a required figure went unretrieved were 0.8% of short runs, 5.8–7.5% of medium runs and 46.7–53.3% of long runs under the two failure conditions, and among them the model cited a missing figure in 51 of 56 and 52 of 64 long runs. Task length therefore appears to act mainly by making unrecovered failures common. These runs probably stayed correct because one misestimated yearly margin is diluted in an eight-year average and falls inside the 0.1-point tolerance, so the tolerance may be part of the explanation as well as the agent; I did not test this. A post hoc simulation of the live design suggests that company-resampling intervals are 6 to 9 points wide even when the true rate is 3%, so the design cannot separate rates of a few percent from zero.

5 Discussion

The scripted controls show that the pipeline separates numerical correctness from its encoded process rules: in stratum A the default-trap policy reached the right answer without retrieving the requested basis, while in stratum B the same choice changed the answer. This verifies the construction of the benchmark and is not evidence about how often an independently developed agent would make that choice. It also shows why the denominator matters, since a policy with no correct answers has no estimable σ even if every run breaks the rule.

The live results add a qualified observation. In a synthetic world where `gpt-4o-mini` was already about 94% accurate and passed the rubric at baseline, tool failures, time pressure, a buried request or a distractor tool on their own produced no silent defect. Defects appeared when a long task met failing tools, and then the model cited figures it had never retrieved. Two silent defects in 2,263 correct runs in the first suite make that a hypothesis for a larger study and not an estimate, and because the cases share only 14 companies, the absence of defects elsewhere should not be read as evidence of safety. The incorrect baseline answers were process-compliant and consistent with

arithmetic error, whereas the process failures arose in a different situation, so an outcome-only check or a process-only check alone would have missed one or the other.

The near-zero baseline rate has three readings that the data cannot fully separate. The model may be reliable in this setting; the checker may miss defects, which the hand-built scenarios make less likely for the encoded rules but a blind audit of real traces has not excluded; or the task may be too easy for defects to arise. The ladder speaks to the third reading for one model, because defects stayed absent at baseline even with 16 retrievals and appeared only once failures were added. It does not address the checker or other models. Stratified samples of 76 traces from the first suite and 40 from the long tier have been drawn for a blind audit, but none has yet been labelled by an independent person.

Process failures among outcome-correct runs have been reported at much higher rates, for example 27% to 78% of benchmark-reported successes in τ-bench by Cao et al. (2026) and a false-pass share of 80.07% of outcome passes by Yu et al. (2026). A rate of this kind belongs to a model together with a rubric, benchmark and task. My rubric covers only three retrieval and citation rules, so a low rate here is expected even for an agent that would fail broader controls, but I have not scored the same traces under a broader rubric and cannot say how much of the gap it explains.

One objection is that grading the outcome is enough, because a fixed path can penalise valid alternatives. I accept part of this: my rubric does not test call order, and the reordering and extra-lookup tests show that it does not penalise alternative paths. It tests whether the evidence the question requires was retrieved and cited. But in stratum A the omission changes no number, so a silent defect there is a process-compliance failure and nothing more, and σ is not a rate of harmful errors. Whether the requirement is worth enforcing depends on whether the evidence trail matters independently of the answer (Yao et al., 2024), which I do not judge here.

6 Limitations and future work

Limitations. The scripted policies stand in for a live model, so the offline validation does not measure language-model reliability. The live evaluation covers one small model, one synthetic world in which baseline accuracy was already high, six conditions of my choosing and two repetitions on the same 100 cases, which are not independent samples of cases. The written plans were not independently registered, the sample sizes rested on a heuristic and not on simulation, and no random sample of live traces has been audited independently. The cases share only 14 companies, so even the company-level analysis is coarse, and the task involves no ledger writes or authorisation checks, so it cannot speak to the accounts-receivable controls that motivate the framework. The correctness tolerance may also let a misestimated figure pass in a long average.

Future work. The first step is the blind human audit of real traces. Next, the live evaluation should be extended to several models from different families and a harder world, with a finer ladder, graded setback severity, a protocol frozen and time-stamped before collection, and case and repetition counts chosen by simulation. The proposed accounts-receivable extension would add payment records, invoice matching and ledger commitments, and would need its own outcome oracle and evidence-timing predicates. A natural intervention there is a pre-commit evidence gate in the form of a verify-and-retry loop: the agent submits, an external check compares the claimed evidence with the logged retrievals, a failed check returns the reason and allows a bounded number of retries, and the case is referred for manual review after that. Similar deterministic pre-action checks have been studied for shell commands and code edits (Althoubi, 2026), and runtime rule enforcement that can intercept an action is described by Wang et al. (2025). I hypothesise that such a loop may lower the silent-defect rate, because an outcome check would pass these runs whereas a check on the trace would not, but this is a conjecture and not a result. It could be tested only where the baseline produces defects, such as the long tier with failing tools, and it should check requirements that the scoring rubric does not rely on exclusively, since a gate that applies the same checks as the scorer would lower measured defects by construction. Rejection rate, recovery after rejection, referral rate and cost should be reported beside $\widehat{\sigma}$. None of these extensions is measured here.

Ethics and broader impact. The experiment used only synthetic company records, and the live evaluation sent only synthetic records and prompts to a commercial model API, at an estimated cost of about 0.37 USD per repetition of the first suite and 1.54 USD for the ladder at assumed list prices. The intended benefit is narrow: scoring procedural compliance apart from outcome correctness

makes visible a plausible, numerically correct answer obtained the wrong way, which an outcome-only check would pass. I see no direct negative application. The main risk is interpretive, since presenting scripted-control results or one small model's results as evidence about financial agents in general would overstate what has been shown. The findings do not constitute approval for financial deployment.

7 Conclusion

The main message is that a correct answer does not show that the required process was followed, and that the two have to be scored separately, with explicit denominators and uncertainty that respects shared sources. I presented a framework for evaluating silent procedural defects, checked it on 600 scripted runs, and applied it to one served model over 2,400 live runs in the first suite and 1,080 in a task-length study. In the first live suite silent defects arose only when all four setbacks were combined, at two among 2,263 correct runs, and the bounds are too wide to pin down a rate. The task-length study found that silent defects rose with task length when tools failed, reaching about a fifth of correct runs (21.5%) at 16 retrievals under injected failures for this model, though accuracy there was low, so the data show the direction of the effect more firmly than its size. In practice, an outcome-only check would have passed every silent run, so procedural compliance needs its own check, and it should be tested under stress, because a clean result on short, reliable tasks says little about long ones. How common and how costly such defects are in financial agents more broadly is a job for a later study.

References

Asaad Althoubi. Look before you leap: Pre-action verification for LLM agents, 2026. arXiv preprint.

Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. Finance agent benchmark: Benchmarking LLMs on real-world financial research tasks, 2025. arXiv preprint.

Hongliu Cao, Ilias Driouich, and Eoin Thomas. Beyond task completion: Revealing corrupt success in LLM agents through procedure-aware evaluation, 2026. arXiv preprint.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *arXiv preprint arXiv:2406.13352*, 2024. URL `https://arxiv.org/abs/2406.13352`.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. URL `https://arxiv.org/abs/2305.20050`.

Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations, 2024. arXiv preprint.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2023. URL `https://arxiv.org/abs/2309.15817`.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback, 2022. arXiv preprint.

Haoyu Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents, 2025. arXiv preprint.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024. URL `https://arxiv.org/abs/2406.12045`.

Boxi Yu, Yuzhong Zhang, Zhenfeng Su, Linfu Xi, Yinghan Yao, Shitao Chen, Yang Cao, Liting Lin, Lionel Briand, and Emir Muñoz. CXBench: A benchmark for process-compliant LLM agents in regulated customer-experience workflows, 2026. arXiv preprint.

A Reproducibility record

The harness separates case construction (`fixtures.py`), policy and API execution (`run_experiment.py`, `backends.py`), compliance scoring (`defects.py`) and statistics (`analyze.py`). The scripted policies used generation seed 0, and all analyses used seed 12345 with 10,000 bootstrap replicates. The live suite was produced by the extended harness retained under `pathmat/results/live_suite_gpt-4o-mini/code_snapshot`, with generation seed 0, temperature 1.0 and served model `gpt-4o-mini-2024-07-18`. That directory holds every trajectory for both repetitions and the written plan. The tolerance correction and the company-level analysis are in the merged pipeline under `pathmat/`, which `paper/build_live_artifacts.py` imports to produce the full two-repetition table (`generated/live_table.tex`). Because the model was sampled at temperature 1.0, a rerun will not return identical trajectories, so the tables are rebuilt from the retained ones.

The task-length study is in `pathmat/ladder/`, with its dated written plan (`PREREGISTRATION.md`), world generator, runner and offline tests; it used generation seed 0, a world of 60 companies over fiscal years 2015–2024 with restatement probability 0.2, and the same model, temperature and bootstrap settings. Table 2 is generated from `pathmat/ladder/out_ladder/ladder_results.json` by `paper/build_ladder_artifacts.py`. The real-trace edits and precision simulation are `pathmat/mutate_real_traces.py` and `pathmat/sim_precision.py`. Blind-audit samples have been drawn (`pathmat/audit_sample.py`, `pathmat/ladder/audit_sample_ladder.py`) but not yet labelled by an independent person. Token cost was measured from the provider's usage fields, and the per-token price used to convert it to dollars is an assumption.